30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 30

Mobile System Design & Integration — From Requirements to Trade-offs



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 30

Why system design is the finale

Senior and architect loops are usually decided here. Nobody asks you to recall an API — you design a feature end to end and defend every trade-off:

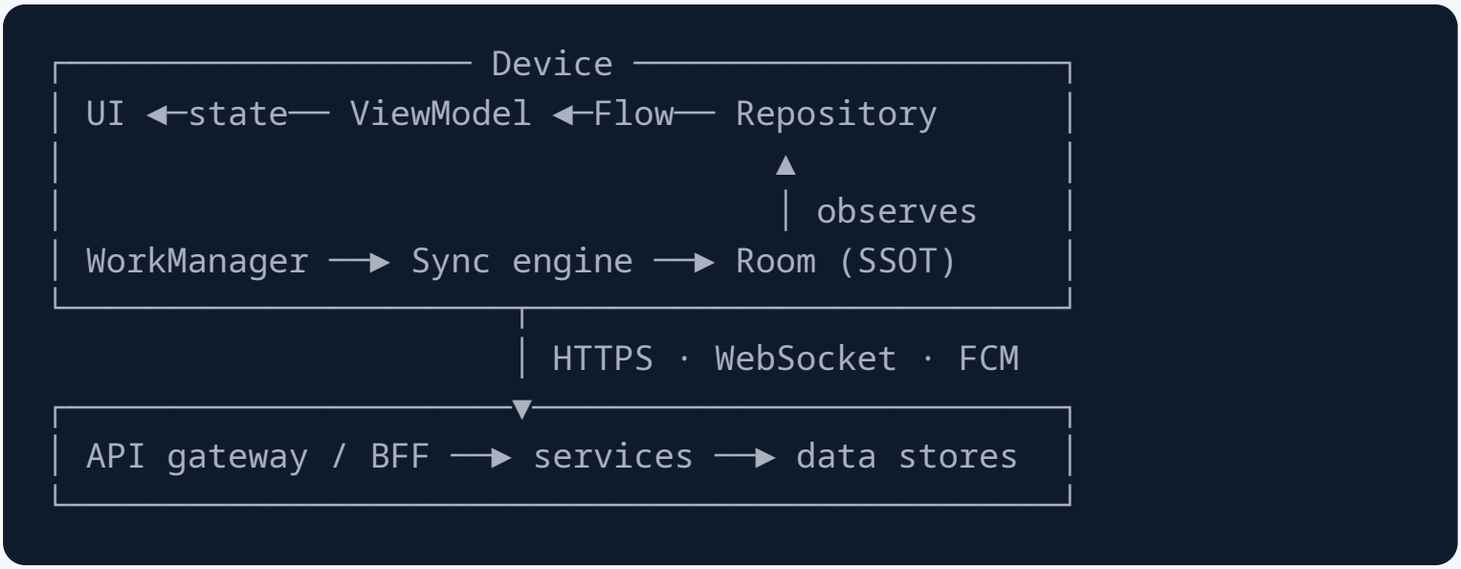
- Mobile design is **not** backend design: the device goes offline, gets killed, has a wrong clock and runs old code for years
- Integration details — retries, idempotency, consistency — are where real production bugs live
- A clear **framework** beats a clever diagram
- Everything from Days 1–29 comes together in one answer

Level	Questions
Junior	The API contract · Client integration patterns · Getting server updates
Mid	Resilience · Consistency & conflicts · Idempotency · Sagas
Senior	The design framework · Design: chat app · Design: offline-first feed
Architect	Design: payments · Gateways, BFF & versioning · Scale, observability & trade-offs



Esmaeeil Moradi
@esmaeeil-moradi700

Mental model: two halves and a contract



- The **device** is an unreliable node of a distributed system — design for it
- Inside the app, data flows one way: network → Room → Flow → UI
- The **contract** (API + events) is the seam between the two halves

WHERE TO SPEND YOUR INTERVIEW TIME

In a mobile round, the backend is a box behind a contract. Go deep on the device side: storage, sync, state and failure modes.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 30

Junior

The API contract · Client integration patterns · Getting server updates



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. Integrating with a complex backend: contract first

Treat the backend as an external system behind a **contract**, and keep all knowledge of it inside the **data layer**. A good contract defines:

- Resources, operations and schemas — in an OpenAPI spec, GraphQL schema or `.proto`
- A **typed error model**, not just an HTTP status and a message string
- Pagination (cursors), auth and token refresh, rate limits (`429` + `Retry-After`)
- Idempotency for writes, versioning and a deprecation policy

Style	Good at	Watch out
REST + JSON	Simple, HTTP/CDN caching, tooling everywhere	Several round trips for complex screens
GraphQL	Client picks fields; one request per screen	HTTP caching is harder; query cost on the server
gRPC	Compact protobuf, streaming, strict schema	Needs HTTP/2; harder to inspect and cache



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. What interviewers check

INTERVIEW TRAP

"GraphQL is simply more efficient for mobile." (v1) — It removes over-fetching, but moves the cost to server resolvers, makes HTTP caching harder, and one badly designed query is as slow as five REST calls. A REST **backend-for-frontend** often gives the same one-call-per-screen benefit.

INTERVIEW TRAP

"A vendor SDK is always simpler than our own API client." — Faster to start, but it adds app size, startup work, permissions and lock-in. Wrap it behind **your own interface** so it can be faked in tests and replaced later.

FOLLOW-UP

"Where does backend knowledge live in the app?" — Only in data sources: endpoints, DTOs, headers. Repositories map DTOs to domain models; a ViewModel never sees a DTO or an HTTP status code.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Client integration patterns in 2026

```kotlin
fun UserDto.toDomain() = User(
    id = UserId(id),
    name = displayName ?: "Unknown",
)

class ProfileRepository(
    private val users: UserApi,
    private val billing: BillingApi,
) {
    suspend fun profile(id: String): Profile = coroutineScope {
        val user = async { users.get(id).toDomain() }
        val plan = async { billing.plan(id).toDomain() }
        Profile(user.await(), plan.await())
    }
}
```

- **Adapter** = mappers like `toDomain()` and wrappers around vendor SDKs; **Facade** = a repository hiding several services
- `coroutineScope`: if one call fails, the other is cancelled — structured concurrency (Day 4)

INTERVIEW TRAP

"Use the Singleton pattern for Retrofit and the database." (v1) — You want **one instance**, not **global access**: let DI scope `OkHttpClient` and Retrofit (`@Singleton` in Hilt). A global `object` hides dependencies and can't be faked in tests.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Getting updates from the server

Mechanism	Use when	Cost
Periodic polling	Data changes rarely; simplest	Wasted requests; latency = interval
SSE / long polling	One-way server → client stream	A held connection per client
WebSocket	Two-way, low latency, app in foreground	Reconnect & resume logic; battery if kept open
FCM push	App in background or not running	Best effort: may be delayed or collapsed; small payload

Inside the app, the event-driven part is simple: the sync layer writes to Room, and every screen observes the repository's `Flow`. **The database is the event bus.**



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. What interviewers check

INTERVIEW TRAP

"Use a global Event Bus so components stay decoupled." (v1) — In 2026 that's an anti-pattern: invisible coupling, no owner for the state, lifecycle leaks, and events lost when nobody listens. Expose state from repositories as `Flow` — one source of truth.

INTERVIEW TRAP

"The push notification delivers the data." — Treat FCM as a **signal**: "something changed, go sync". Messages can be delayed or collapsed, and a force-stopped app receives none. The truth always comes from the API.

FOLLOW-UP

"Can we keep the WebSocket open in the background?" — See Q9: connect while the UI is visible, and rely on FCM when it isn't.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 30

Mid-Level

Resilience · Consistency & conflicts · Idempotency · Sagas



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Resilience: retries done right

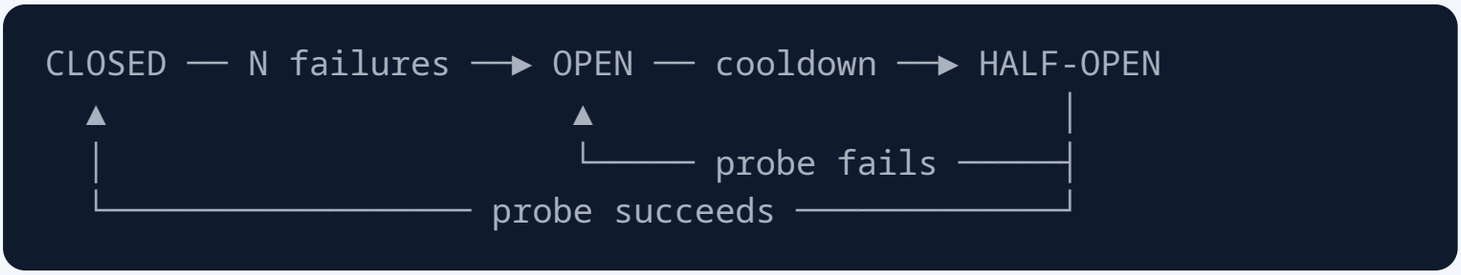
```kotlin
suspend fun <T> withRetry(
    maxAttempts: Int = 4,
    baseDelayMs: Long = 500,
    maxDelayMs: Long = 16_000,
    block: suspend () -> T,
): T {
    var backoff = baseDelayMs
    repeat(maxAttempts - 1) {
        try {
            return block()
        } catch (e: IOException) {
            // transient network error: retry below
        }
        // jitter: wait a random time in [backoff/2, backoff]
        delay(Random.nextLong(backoff / 2, backoff + 1))
        backoff = (backoff * 2).coerceAtMost(maxDelayMs)
    }
    return block() // last attempt: errors propagate
}
```

- Retry only **transient** failures (I/O, 5xx, `429` honoring `Retry-After`) and only **safe** requests — idempotent, or with an idempotency key (Q6)
- `CancellationException` isn't caught, so cancellation still works



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. The fleet view: circuit breakers and fallbacks



- **Fallbacks:** cached data with "updated 5 min ago", hide non-critical features, queue writes in an outbox
- **Server-side levers:** `Retry-After`, remote-config kill switches, a minimum supported app version

INTERVIEW TRAP

"A client circuit breaker gives the failing service time to recover." (v1) — One phone barely matters. The danger is the **fleet**: millions of apps retrying in lockstep after an outage. Jitter, `Retry-After` and server-controlled backoff protect the backend.

INTERVIEW TRAP

Retrying in every layer — OkHttp, an interceptor, the repository and WorkManager: 3 × 3 × 3 = 27 requests per user action. Retry in **one** place.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Consistency: optimistic UI and conflicts

Across devices and services, data is **eventually consistent**. Update the UI optimistically, then let the server decide:

```
suspend fun rename(note: Note, title: String) {
    dao.upsert(note.copy(title = title, sync = PENDING))
    try {
        val saved = api.rename(note.id, title, note.etag)
        dao.upsert(saved.toEntity())          // server copy wins
    } catch (e: HttpException) {
        if (e.code() == 412) dao.markConflict(note.id)
        else dao.upsert(note)                 // roll back
    } catch (e: IOException) {
        outbox.enqueueRename(note.id, title) // retry later
    }
}
```

- `note.etag` goes out as an `If-Match` header — **optimistic concurrency**: a stale write gets `412 Precondition Failed` instead of silently overwriting



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Choosing a conflict strategy

Strategy	How	Trade-off
Last write wins	Newest timestamp overwrites	Silent data loss; device clocks drift
Version check	ETag / version; stale writes rejected	Client must merge or ask the user
Field-level merge	Merge non-overlapping field edits	Needs per-field change tracking
CRDTs	Data types that always merge	Complexity and metadata overhead

INTERVIEW TRAP

"Last write wins: the last change received is saved." (v1) — Arrival order isn't edit order: a phone that was offline for a day syncs and overwrites a newer edit. Use **server-assigned versions**; if you pick LWW, know you're choosing data loss.

FOLLOW-UP

"Optimistic or pessimistic UI?" — Optimistic for cheap, reversible actions (like, rename, send message). Pessimistic — show progress, wait for the server — for money, irreversible or security actions.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Idempotency: making retries safe

`GET`, `PUT` and `DELETE` are idempotent by definition; `POST` and `PATCH` are not. For non-idempotent writes, send an **idempotency key** — one per user intent:

```
interface OrderApi {
    @POST("orders")
    suspend fun create(
        @Header("Idempotency-Key") key: String,
        @Body body: CreateOrderDto,
    ): OrderDto
}

// One key per user intent, persisted with the operation
val op = PendingOrder(key = UUID.randomUUID().toString(), cart)
outboxDao.insert(op)                 // survives process death
api.create(op.key, op.cart.toDto())  // retries reuse op.key
```

The server stores `key → first result` and returns the same result for repeats — a retried request can't create a second order.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. What interviewers check

INTERVIEW TRAP

"We add the key in an OkHttp interceptor with a fresh UUID." — Every retry gets a **new** key, so the server sees new requests. The key must be created once per user intent and stored with the pending operation.

INTERVIEW TRAP

"Our queue delivers exactly once." — Networks give at-most-once or at-least-once. **Exactly-once effect** = at-least-once delivery + idempotent processing + dedupe on receive.

FOLLOW-UP

"Whose job is idempotency?" — Both sides. The client generates and persists the key; the server stores it with the result and rejects the same key with a different body.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Multi-step transactions: sagas

```
Order saga — orchestrated by the backend

 step                   compensation if a later step fails
 1 reserve stock        release stock
 2 authorize payment    void the authorization
 3 create shipment      cancel shipment
 4 capture payment      — last step

 step 3 fails → void (2) → release (1) → order FAILED
```

- A saga = local transactions + a **compensation** for each; no global lock
- **Orchestration** (one coordinator) is easier to observe for payments; **choreography** (services react to events) couples less
- Order matters: reserve and authorize first, **capture money last —** v1 charged the card first, forcing refunds on every stock failure



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Where the saga runs

The phone sends **one** command — `POST /orders` with an idempotency key — and observes the order's status (`PENDING → CONFIRMED` or `FAILED`) via push or polling.

INTERVIEW TRAP

"The app calls payment, then inventory, then shipping." — The device can be killed, go offline or be tampered with between steps. **Never make the client the coordinator** of a distributed transaction.

INTERVIEW TRAP

"Compensation is a rollback." — It's a new, visible business action (refund, release, apology email), and sometimes impossible. Design each step so a compensation exists — and make it idempotent too.

FOLLOW-UP

"Why not two-phase commit?" — It blocks, the coordinator is a single point of failure, and third-party providers (payment, shipping) don't take part in your 2PC.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 30

Senior

The design framework · Design: chat app · Design: offline-first feed



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. "Design X for Android" — the framework

#	Step	What you produce
1	Requirements	Functional, non-functional, **out of scope**
2	API contract	Endpoints/events, pagination, errors
3	Data model & storage	Entities, what's cached where, for how long
4	Sync & offline	Read path, write path, conflicts
5	UI state & architecture	Layers, modules, one state per screen
6	Scaling concerns	Battery, data, memory, big lists, low-end devices
7	Observability	Metrics, logs, traces, alerts
8	Trade-offs	What you chose, what you rejected, why

DRAW EARLY

A box diagram of the client layers and the backend in the first ten minutes gives the interviewer something to point at — and gives you a map for the rest.



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Requirements first

Questions worth asking before drawing anything:

- Which actions must work **offline**? How fresh must data be?
- Real-time or "within a minute"? Push when the app is closed?
- How costly is a wrong or duplicated result — likes, or money?
- Media? Sizes? Poor networks or low-end devices in target markets?
- Privacy: PII, end-to-end encryption, compliance?

INTERVIEW TRAP

Answering with backend infrastructure — load balancers, sharding, replication — as v1 did. In a **mobile** round that's the wrong depth: keep the backend at contract level unless asked, and go deep on the client.

INTERVIEW TRAP

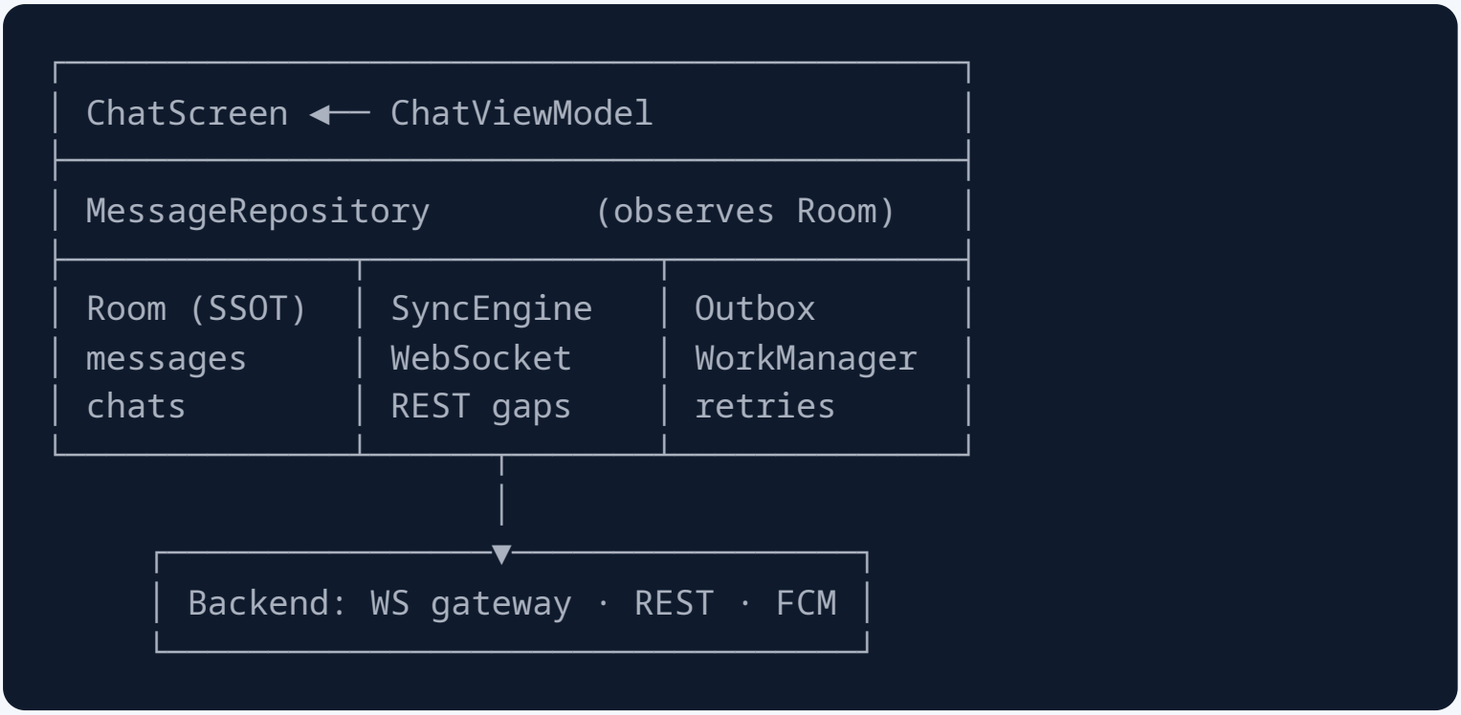
Opening with libraries — "Retrofit, Hilt, Room". Tools come after requirements; the interviewer scores the reasoning.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Design a chat app — requirements & shape

Functional: 1:1 and group chats, text and images, delivered/read receipts, history, send while offline, notifications. **Non-functional:** no lost or duplicated messages, stable order, instant in foreground, battery-friendly in background. **Out of scope:** calls, end-to-end encryption (follow-up).





Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Contract and data model

```
POST /v1/chats/{id}/messages      body carries clientMsgId
GET  /v1/chats/{id}/messages?beforeSeq=…      history page
GET  /v1/sync?since=<cursor>                  missed events
WS   message.new · message.ack · receipt.read
FCM  data { chatId }  → "go sync", never the message body
```

```kotlin
@Entity(
    tableName = "messages",
    indices = [Index(value = ["chatId", "seq"])],
)
data class MessageEntity(
    @PrimaryKey val clientMsgId: String, // UUID from sender
    val serverId: String?,     // null until the server acks
    val chatId: String,
    val seq: Long?,            // server order; null = pending
    val senderId: String,
    val body: String,
    val status: MessageStatus, // PENDING, SENT, READ, FAILED…
    val createdAtLocal: Long,  // display only, never ordering
)
```

Query: `ORDER BY seq IS NULL, seq, createdAtLocal` — confirmed messages in server order, pending ones at the bottom.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Send and receive paths

Send:

1. Insert a `PENDING` row with a new `clientMsgId` — the UI shows it instantly
2. The socket (or the outbox worker when offline) sends it; `clientMsgId` is the idempotency key
3. The server assigns `seq` and acks → the row becomes `SENT`; a duplicate send gets the same ack

Receive:

1. Foreground: WebSocket event → upsert by `clientMsgId` (dedupe for free)
2. Gap in `seq` (have 38, got 41) → `GET /sync?since=…` fills it
3. Background: FCM high-priority data message → short sync → notification
4. Reconnect with backoff + jitter; resume from the last cursor

> **INTERVIEW TRAP**
>
> *"Order messages by the device timestamp."* — Clocks differ and users change them. Order by the **server sequence** per chat; the local time is only for display.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Trade-offs and follow-ups

Decision	Chosen	Rejected — why
Real-time	WebSocket while visible + FCM	Polling: latency and battery
Delivery	At-least-once + dedupe	"Exactly once": doesn't exist
History	Room + Paging (`RemoteMediator`)	Loading all messages into memory
Media	Upload first (resumable), then send the media id	Base64 inside the message

INTERVIEW TRAP

"Keep the socket alive in a foreground service for background delivery." — It drains the battery, and foreground service types (required since Android 14) must match a user-visible task — an idle listener isn't one. Use FCM high-priority messages that lead to a visible notification.

FOLLOW-UP

"Add end-to-end encryption?" — The server can no longer read, search or build previews; push carries only an id and the device decrypts before notifying. Key management and multi-device become the hard part.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Design an offline-first feed — requirements & API

Functional: infinite scroll, pull to refresh, like and save, images, opens instantly with the last content. **Non-functional:** content on the first frame from cache, smooth scrolling, low data use, no duplicates while paging.

```
GET /v1/feed?cursor=<opaque>&limit=20
→ { "items": [...], "nextCursor": "c_8f3a" }

PUT    /v1/posts/{id}/like        idempotent set-state
DELETE /v1/posts/{id}/like
```

INTERVIEW TRAP

"Page with `?page=3`." — On a live feed, new posts shift offsets: users see **duplicates and skipped items**. Use opaque, server-issued cursors.

INTERVIEW TRAP

"`POST /posts/{id}/toggle-like`." — A double tap plus one retry flips it twice. Model **state** (`PUT` / `DELETE`), which is idempotent by definition.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Room as the source, Paging as the window

```
@OptIn(ExperimentalPagingApi::class)
class FeedRepository(
    private val db: FeedDatabase,
    private val api: FeedApi,
) {
    fun feed(): Flow<PagingData<Post>> = Pager(
        config = PagingConfig(pageSize = 20),
        remoteMediator = FeedRemoteMediator(db, api),
        pagingSourceFactory = { db.postDao().pagingSource() },
    ).flow.map { page -> page.map { it.toDomain() } }
}

// In FeedRemoteMediator: don't refresh while cache is fresh
override suspend fun initialize(): InitializeAction =
    if (cacheIsFresh()) InitializeAction.SKIP_INITIAL_REFRESH
    else InitializeAction.LAUNCH_INITIAL_REFRESH
```

- `RemoteMediator` writes each page and its cursor in **one** Room transaction; the UI only reads Room
- `cachedIn(viewModelScope)` in the ViewModel keeps pages across rotation (Day 14 covers the mediator)



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Writes and scale on the device

- **Likes:** optimistic local update + outbox; the server returns the canonical count
- **Images:** request size-appropriate variants from the CDN; memory + disk cache; prefetch the next page
- **Storage:** cap the cache (e.g. last N posts or 7 days) and evict on refresh
- **Freshness:** show a "New posts" pill instead of moving the list under the user's thumb
- **Ranking** happens on the server; the client renders the order it receives

INTERVIEW TRAP

"The ViewModel keeps the feed in a list." — Lost on process death, two sources of truth, and memory grows with every page. Room + Paging keeps a bounded window.

FOLLOW-UP

"A post was deleted on the server — how does it disappear?" — Tombstones in a delta sync (`since=cursor`), or the refreshed page replaces the old rows; opening it handles `404` / `410` gracefully.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 30

Architect

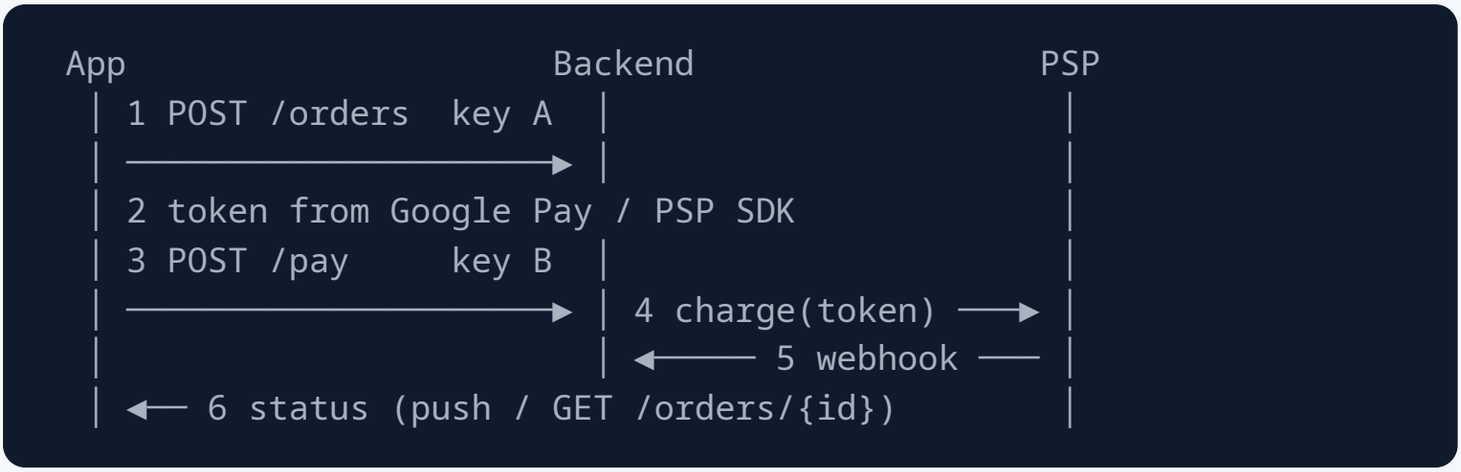
Design: payments · Gateways, BFF & versioning · Scale, observability & trade-offs



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Design a payment flow — requirements & trust

- **Never double-charge, never lose the outcome** — even with process death or network loss mid-request
- Card data never touches your servers or logs: Google Pay or the provider's SDK returns a **token**
- The **server** is the source of truth; the client can be modified



- **PSP** = payment service provider (Stripe, Adyen...). Its **webhook** to your backend is the final word, not the app's callback.
- Two user intents, two idempotency keys: A for "create order", B for "pay this order".



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. A payment is a state machine

```
CREATED → SUBMITTING ─┬─▶ SUCCEEDED
                      ├─▶ REQUIRES_ACTION (3DS) → …
                      ├─▶ FAILED (declined)
                      └─▶ UNKNOWN (timeout, crash)
                              → reconcile: GET status
```

```kotlin
suspend fun pay(orderId: String, token: String): PaymentStatus {
    val attempt = dao.openAttempt(orderId)
        ?: PaymentAttempt(
            idempotencyKey = UUID.randomUUID().toString(),
            orderId = orderId,
            status = PaymentStatus.SUBMITTING,
        ).also { dao.insert(it) }      // persist before sending
    val status = try {
        api.pay(orderId, token, attempt.idempotencyKey).status
    } catch (e: IOException) {
        PaymentStatus.UNKNOWN           // NOT failed: reconcile
    }
    dao.updateStatus(attempt.idempotencyKey, status)
    return status
}
```

- `openAttempt` returns the unfinished attempt for this order, so a retry after a crash or timeout **reuses its key**
- `UNKNOWN` triggers reconciliation (`GET /orders/{id}`), never an automatic "failed" screen



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. What interviewers check

INTERVIEW TRAP

"The request timed out — show 'Payment failed, try again'." — The charge may have succeeded. A timeout is **UNKNOWN**: show "Confirming your payment…", query the order status, and let any retry reuse the **same** idempotency key.

INTERVIEW TRAP

"The SDK callback says success, so unlock the content." — The client is untrusted. Unlock only when the **server** confirms (provider webhooks, server-side verification).

- 3DS sends the user to a bank app; your process may die meanwhile. On return, restore the persisted attempt and reconcile.
- Show the final result as **state**, not a one-off event (Day 3).

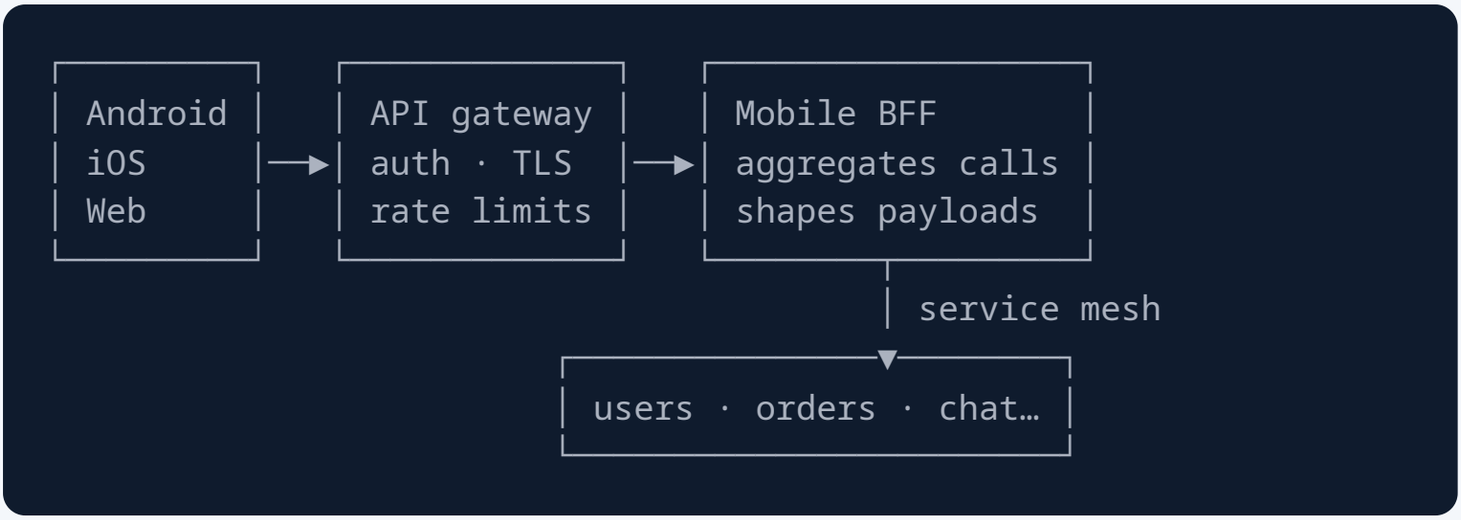
FOLLOW-UP

"Digital goods?" — In a Play-distributed app they generally must use Google Play Billing (regional alternatives exist). Verify purchase tokens on your server and acknowledge purchases — unacknowledged ones are refunded automatically.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Integration at scale: gateway, BFF, mesh



- **API gateway:** one public entry point — auth, TLS, rate limiting, routing
- **BFF:** an API shaped for one client type — ideally one call per screen, owned by or close to the mobile team
- **Service mesh:** service-to-service traffic only; the app never sees it (v1 got this one right)

INTERVIEW TRAP

"The app calls a dozen microservices directly." — Many round trips on mobile latency, every internal service exposed publicly, and the app breaks whenever the backend reorganizes.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Versioning clients you can't redeploy

Old app versions stay installed for **years**. The server must stay backward compatible:

```
val json = Json {
    ignoreUnknownKeys = true  // a newer server adds fields
    coerceInputValues = true  // unknown enum value -> default
}

enum class OrderStatus { PENDING, PAID, SHIPPED, UNKNOWN }

@Serializable
data class OrderDto(
    val id: String,
    val status: OrderStatus = OrderStatus.UNKNOWN,
)
```

- **Additive changes only;** never rename or repurpose a field
- Breaking-change checks in CI: OpenAPI diff, GraphQL schema checks, `buf breaking`; contract tests (Pact)
- Forced update below a minimum supported version (Day 24) — the last resort

INTERVIEW TRAP

A plain `enum` with the default `Json`: a new value like `"REFUNDED"` throws a `SerializationException` in every old client.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Scaling and observability on the client

Concern	Design answer
Battery	Batch work, WorkManager constraints, no background socket
Data	Delta sync (`since=cursor`), compression, image variants
Storage	Cache caps and eviction; tested migrations
Low-end devices	Paging, image downsampling, device-tier features
Fleet effects	Jittered schedules, `Retry-After`, kill switches, staged rollout

Observability — define metrics for the design itself:

- Chat: send-to-ack latency, messages stuck in `PENDING`, reconnect rate
- Feed: time to first content, scroll jank, cache hit rate
- Payments: share of `UNKNOWN` outcomes, time to reconcile
- Send a W3C `traceparent` header so app spans join backend traces (OpenTelemetry); no PII in logs



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Closing the interview: trade-offs

Question	Option A	Option B
Real-time	WebSocket: fresh, costly	Push + sync: cheap, delayed
Conflicts	LWW: simple, loses data	Versions/merge: safe, more work
API	REST + BFF: cacheable	GraphQL: flexible, harder caching
UI	Optimistic: fast, needs rollback	Pessimistic: slow, certain
Cache	Cache-first: instant, may be stale	Network-first: fresh, weak offline

INTERVIEW TRAP

"There's one right design." — Interviewers score reasoning. State your assumptions, and when they change a requirement, **change the design** out loud.

SENIOR ANSWER IN ONE LINE

"Name the requirement that forces each decision, say what you gave up, and say how you'd know in production that you were wrong."



Esmaeeil Moradi
@esmaeeil-moradi700

Day 30 — cheat sheet

- Framework: requirements → API contract → data & storage → sync/offline → UI state → scaling → observability → trade-offs
- The backend is a box behind a contract; go deep on the device.
- Room is the single source of truth; the network writes into it, screens observe `Flow`.
- Retries: transient errors only, backoff + jitter, honor `Retry-After`, in **one** layer.
- Exactly-once doesn't exist on networks: at-least-once + idempotency keys + dedupe.
- Conflicts: server versions (`If-Match` + ETag) beat device-clock last-write-wins.
- Real-time: WebSocket while visible; FCM as a signal in background, then sync.
- Order by server sequence, never by device timestamp. Page with cursors, not offsets.
- Payments: timeout = UNKNOWN → reconcile; the server confirms, never the client.
- Old app versions live for years: additive API changes, unknown-enum fallback, contract checks in CI.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 30

30 days in one slide

Days	What we covered
1–3	Lifecycle & process death · Fragments & Navigation · Architecture
4–5	Coroutines · Flow
6–8	Jetpack essentials · Dependency injection · Compose
9–10	Testing: unit, UI, screenshots, benchmarks
11–12	Networking · Network security & auth
13–14	Data layer · Offline-first & sync
15–16	Background work · Services, alarms & push
17–18	Performance: jank, memory, app size, battery
19–20	Security: storage, Play Integrity, passkeys
21–22	Gradle & build performance
23–24	CI/CD · Releases, rollouts & feature flags
25–26	Modularization · Advanced Kotlin
27–29	Architecture, performance & global scale
30	System design & integration



Esmaeeil Moradi
@esmaeeil-moradi700

Thank you

Version 2 exists because of you. After version 1, you asked for fewer slides and more depth, real code instead of theory, the traps interviewers actually push back on, and an update for today's Android. Every day of this edition was rebuilt around that feedback.

To everyone who followed version 2 for thirty days — who saved a post, answered a self-check in the comments, argued with a trap, or sent a carousel to a friend preparing for an interview: **thank you.** This series was ours, not mine.

ONE LAST PIECE OF ADVICE

Don't memorize these answers. Understand them well enough to rebuild them from first principles — that's what interviewers are really looking for.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 30

Let's keep talking

- **Which day helped you most** — and which topic deserves a deep dive next? Tell me in the comments.
- **Preparing for a system design round?** Comment with the app you'd like to see designed.
- **Save the whole series** — all 30 days are on my profile, in English and Persian.
- **Connect with me** on LinkedIn and share your interview wins — I read every message.

Keep learning, keep building — and good luck in your next interview.



Esmaeeil Moradi
@esmaeeil-moradi700